

Threat Detection in Network Traffic using Time Series Analysis



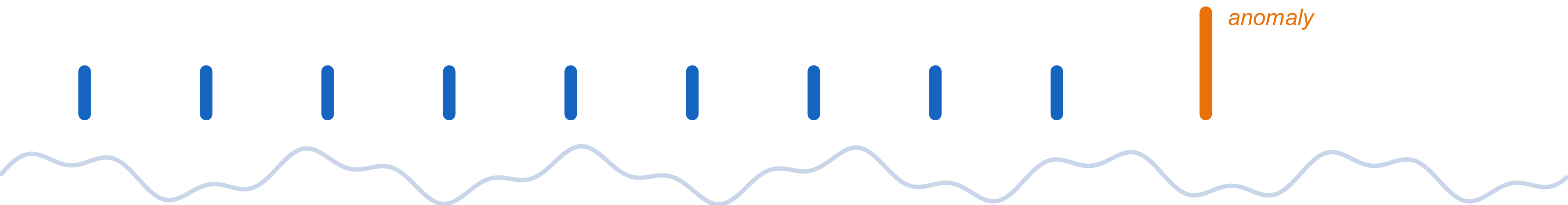
Dissertation thesis defence

Josef Koumar

Supervisor: Assoc. Prof. Tomáš Čejka, Ph.D.

Department of Digital Design, Faculty of Information Technology, CTU in Prague

7 October 2026



Encryption hides the payload – not the behaviour

WHAT A PACKET LOOKS LIKE TODAY:



- ⊘ **Deep packet inspection and signature IDS**
(Snort, Suricata) cannot see the content anymore.
Decrypting proxies are not possible at ISP scale.

WHAT REMAINS OBSERVABLE

- 🕒 Timestamps of packets and flows
- 📦 Sizes and counts of packets / bytes
- 🔄 Direction and who talks to whom
- 📶 Repetition and regularity over time

= a **time series** describing the behaviour of the traffic



Threats hide in encrypted channels – but they still behave differently:

Botnet C&C beaconing

Cryptomining

Tunnels (VPN, TOR, DoH)

DoS

Brute force

Exfiltration

Problem statement

How can we detect security threats in **high-speed encrypted traffic** using only **time series of metadata** and **deploy it in a real-world (ISP) network**?



Research gap

- SOTA feeds the first N packets (SPLT) into deep learning, while time series analysis itself is rarely used.
- Long-term behaviour (periodicity, trends) is lost when traffic is aggregated into flows.
- Supervised models need labels and traffic characteristics drifts over time.



Deployability gap

- Methods are evaluated offline, rarely under real constraints.
- Large or high-speed networks (like ISP) limits telemetry size, memory and processing time.
- Too many alerts overwhelm the security team (alert fatigue).

0 B

of payload visible to the monitoring

400 Gbps

links in the CESNET3 backbone

> 1 M

flow records per second in a large ISP

Goals of the thesis

 **Overall goal:** use time series analysis of network traffic for **deployable** detection of security threats

G1



Time series

Create and describe time series from network traffic

G2



Detection

Research, design and develop threat detection using time series analysis

G3



Deployability

Evaluate the deployability of the proposed detection methods

G4



Open data

Create and publish network traffic time series datasets for the community

Badges in the top-right corner of each following slide show which goals it addresses.

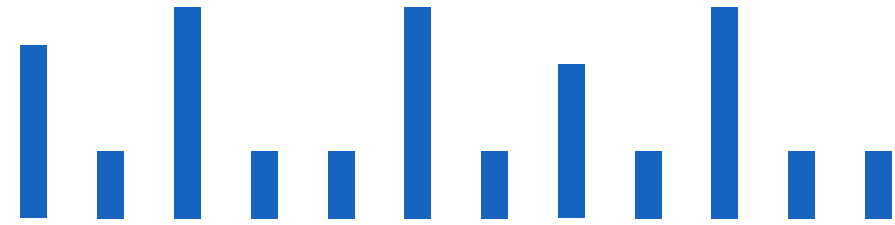
Three time series views of the same traffic

Time horizon: **seconds – minutes**

Time horizon: **hours – days**

Time horizon: **weeks – months**

Single flow time series



DATA POINT

Packets of one connection

REVEALS

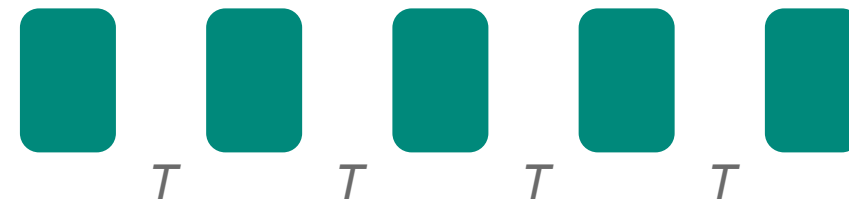
How one connection behaves

METHOD

Supervised classification

→ NetTiSA flow

Flow time series



DATA POINT

Flows between a host and a service

REVEALS

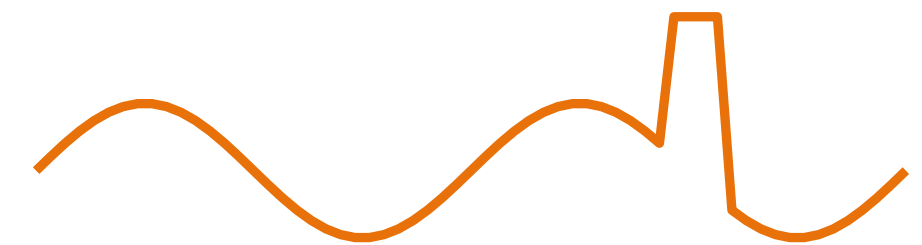
Automated, periodic communication

METHOD

Periodicity detection

→ Periodic features

Aggregated time series



DATA POINT

Traffic volume per IP / subnet

REVEALS

Deviation from normal behaviour

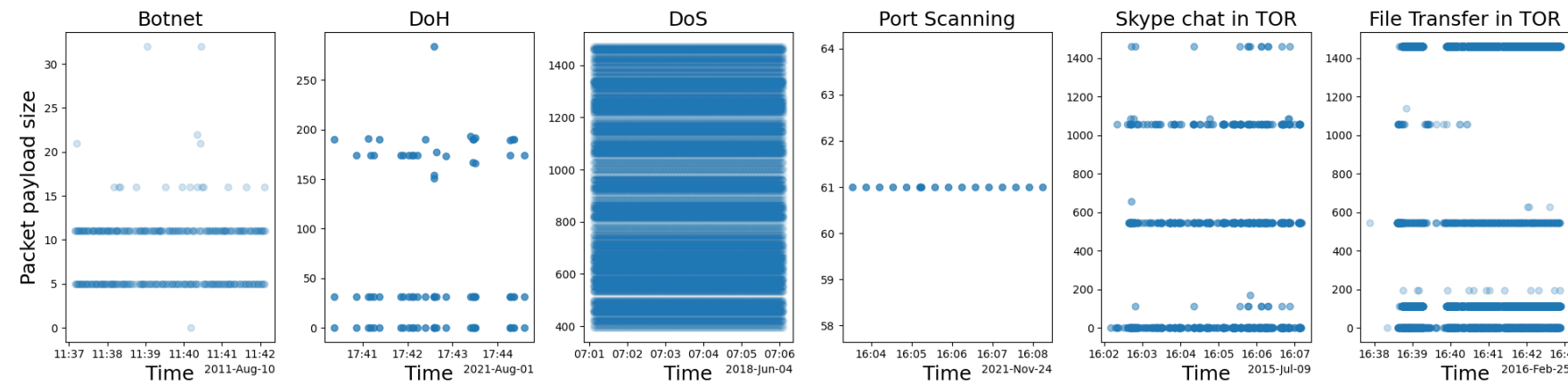
METHOD

Unsupervised anomaly detection

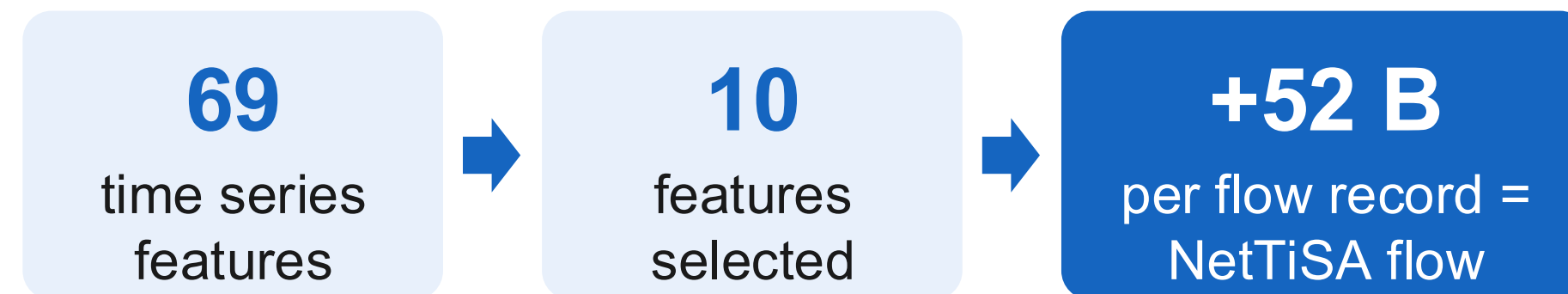
→ Forecasting + LLM Explainability

Single flow time series → NetTiSA flow

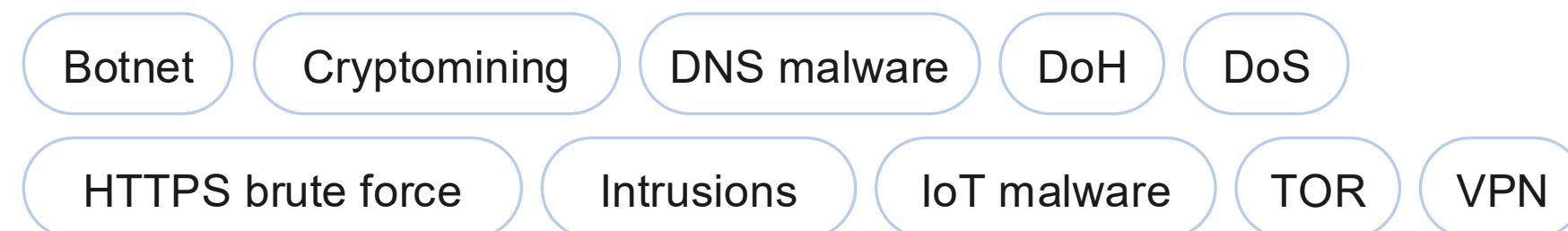
Threats have distinct fingerprints inside one connection



Packet sizes over time: botnet, DoH, DoS, port scan, TOR



Evaluated on 10 detection tasks:

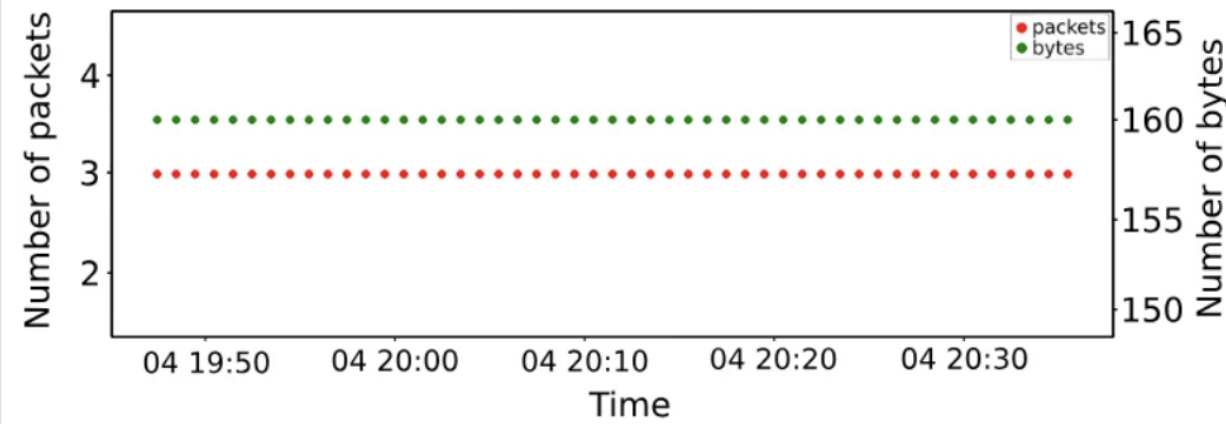


Compared with the best related work number of datasets where NetTiSA is ...



Similar accuracy to methods tailored to one threat with one universal, compact feature set.

Flow time series → periodicity detection

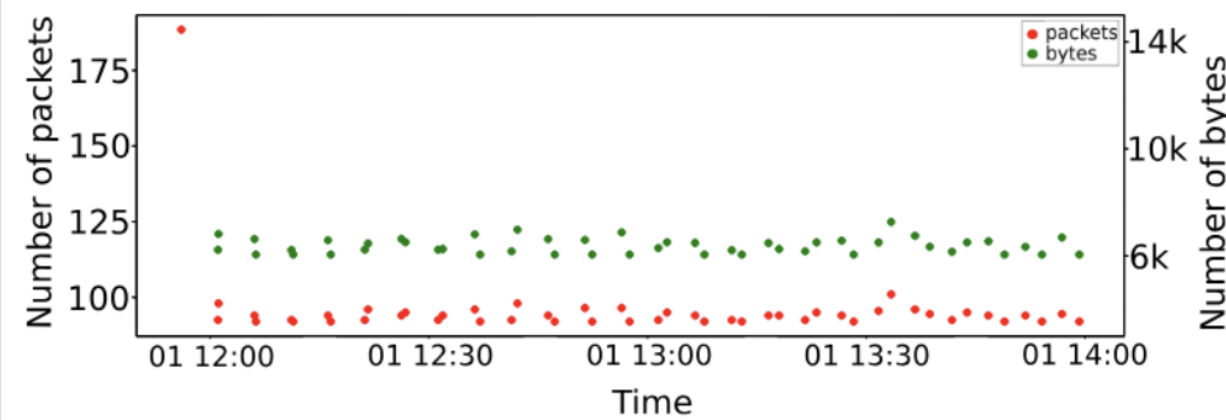


Clear
constant volume,
e.g. botnet
heartbeat

90 %

F1-score

Classification of applications,
services and operating systems



Sinusoidal
varying volume,
e.g. app
synchronisation

92 %

F1-score

Cryptomining detection
(accuracy up to 95 %)

100 %

precision

Botnet C&C detection across
multiple datasets



Lomb–Scargle periodogram on unevenly spaced
flows → periodic features → ML classifier

Aggregated time series → anomaly detection



CESNET-TimeSeries24 – novel real-world ISP dataset

40 weeks of CESNET3 traffic · 10 min, 1 h and 1 day aggregation

275,124

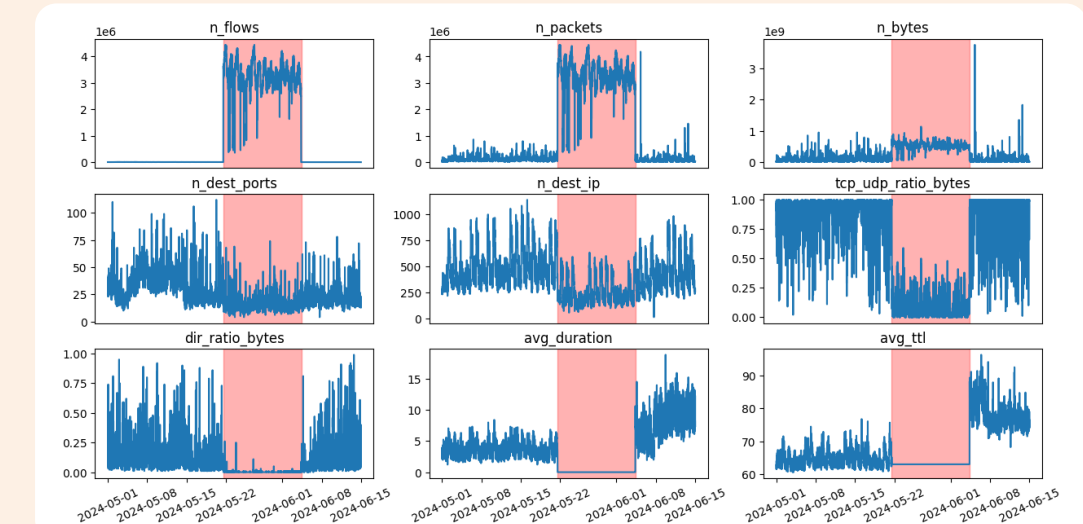
IP addresses

548

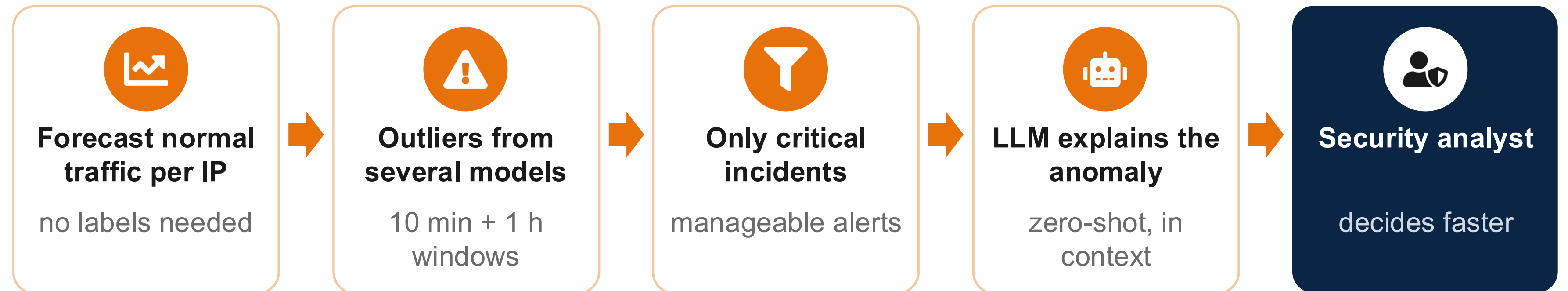
subnets

283

institutions



Anomaly detection system (NODS) + LLM explanation



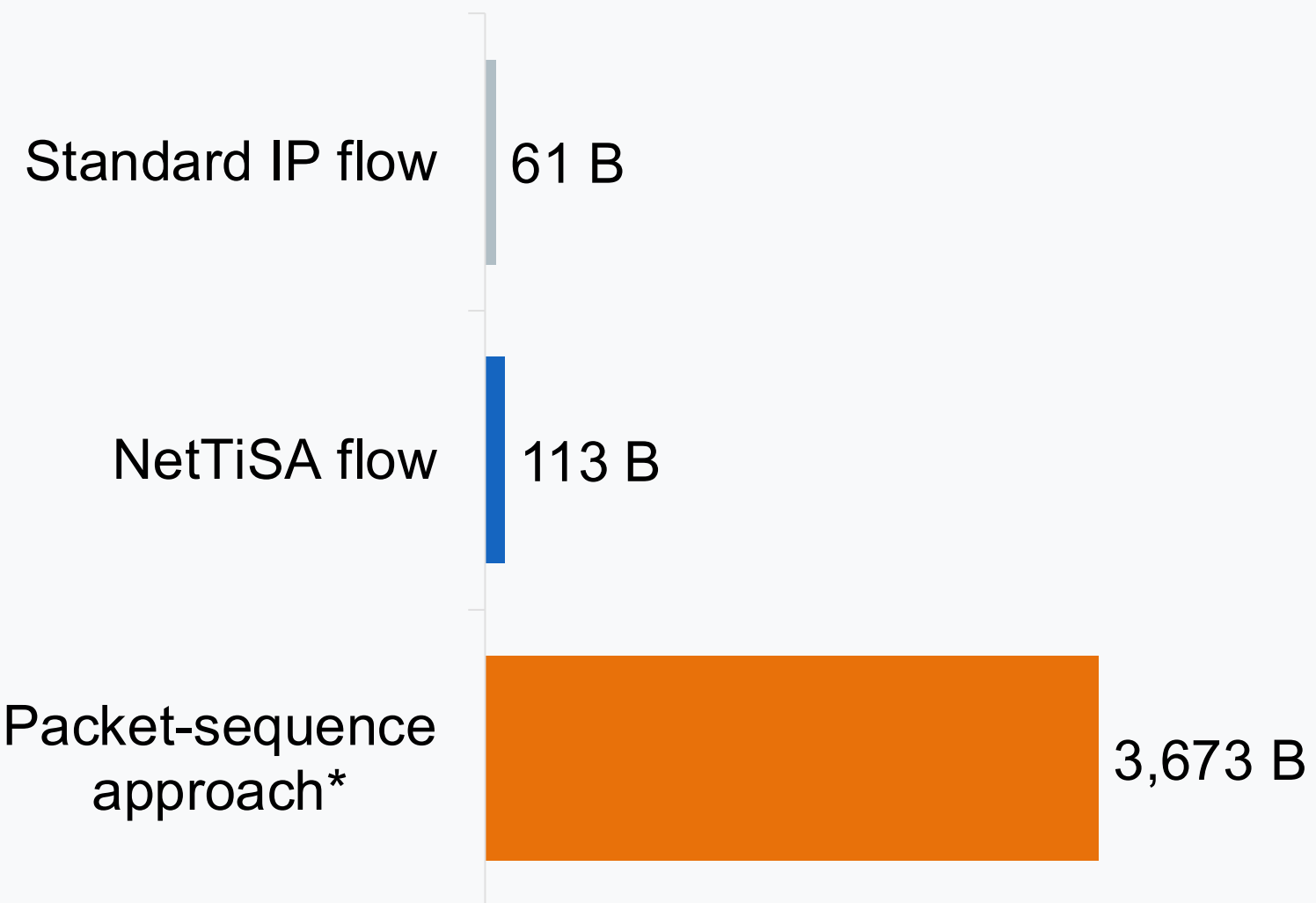
Deployed in the CESNET3 network – finds deviations, including unseen threats, without any attack labels.

Deployable at ISP scale



Telemetry per flow record

bytes exported for every flow



NetTiSA fits into a 1 Gbps line even at 1 M flows/s.

** best related work on the VNAT dataset*



NetTiSA FLOW

In production in CESNET2

8 probes, ~200,000 flows/s · no slowdown at 100 Gbps · 60 B memory per flow



PERIODICITY DETECTION

3 million flows in 5 seconds

Integrated into the DP3 platform, alerts feed the DeCrypto cryptomining detector



ANOMALY DETECTION (NODS)

Deployed on CESNET3 (1–400 Gbps)

One VM (8 CPU, 16 GB RAM) · only critical incidents reach the SOC · LLM explainability is still experimental prototype

Open data for the community

9



datasets on Zenodo
+ 1 dataset on GitHub

33,451

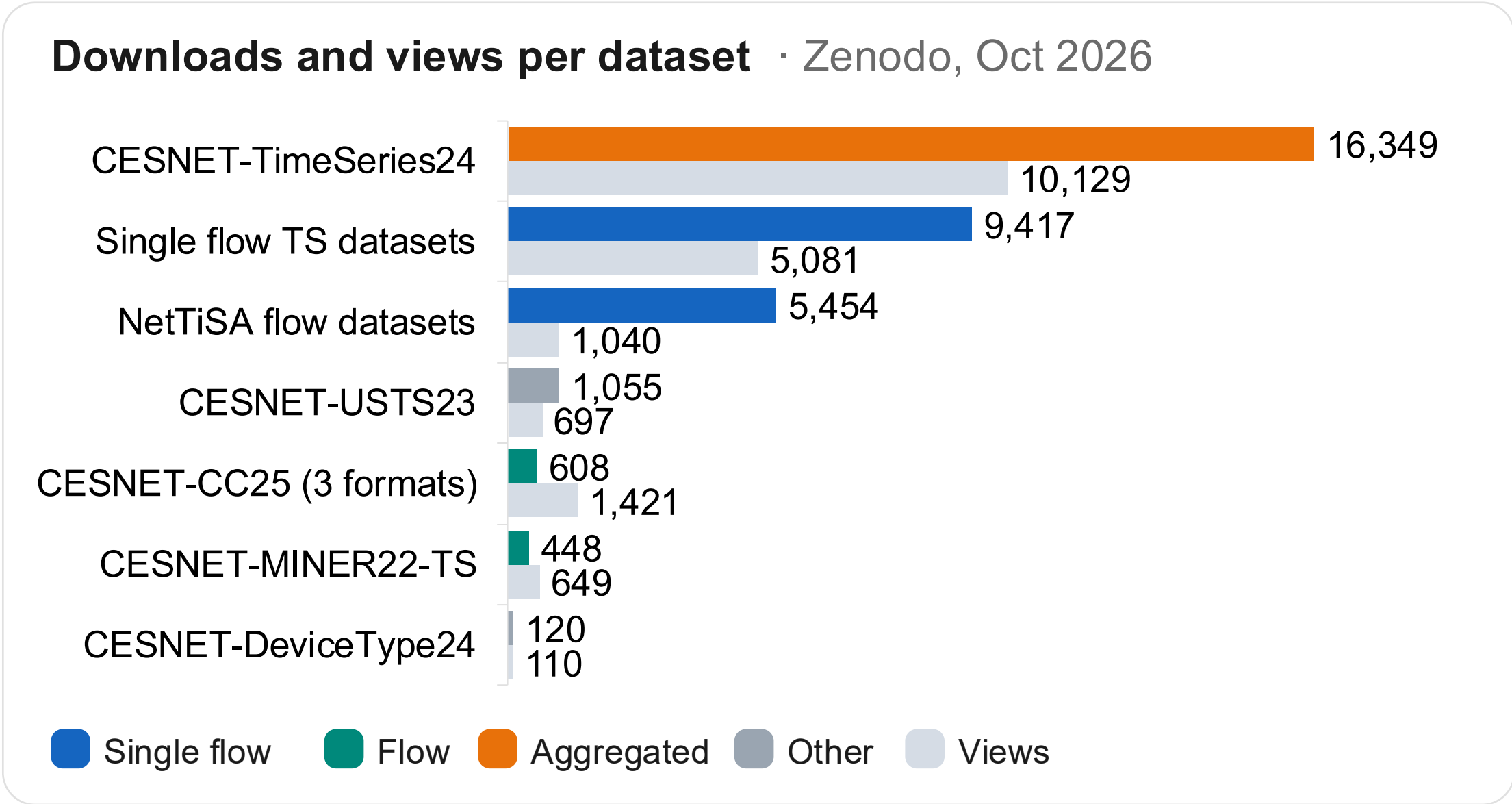


downloads
19,127 views on Zenodo

1



library: CESNET-TSZoo
reproducible analysis



Why it matters

-  Real ISP traffic from CESNET – not only lab captures
-  Source code of experiments released with the papers
-  CESNET-TimeSeries24 paper: 60 citations (Nature Scientific Data)
-  Published under the ethics statement approved by the CTU Ethics Committee

Summary: all four goals fulfilled



G1	Time series	Taxonomy of three network time series: single flow, flow and aggregated	TMA 2023	
G2	Threat Detection	Three detection methods: NetTiSA features (10 tasks, SOTA level) · periodicity detection (C&C 100 % precision) · forecasting-based anomaly detection with LLM explanation	CNSM ×4 · Computer Networks · NOMS ×2	
G3	Deployability	All methods deployed or integrated: NetTiSA in CESNET2 production · periodicity in DP3 · NODS on CESNET3	tested up to 100 Gbps	
G4	Open data	Datasets CESNET-TimeSeries24, CESNET-CC25, CESNET-MINER22-TS, CESNET-USTS23, CESNET-DeviceType24, Source code of experiments released + CESNET-TSZoo library	Nature Scientific Data · Zenodo	

Impact of the research

18



peer-reviewed papers

3 Q1 journals · 15 Core B conferences

221



citations

Google Scholar
h-index 8

9



open datasets

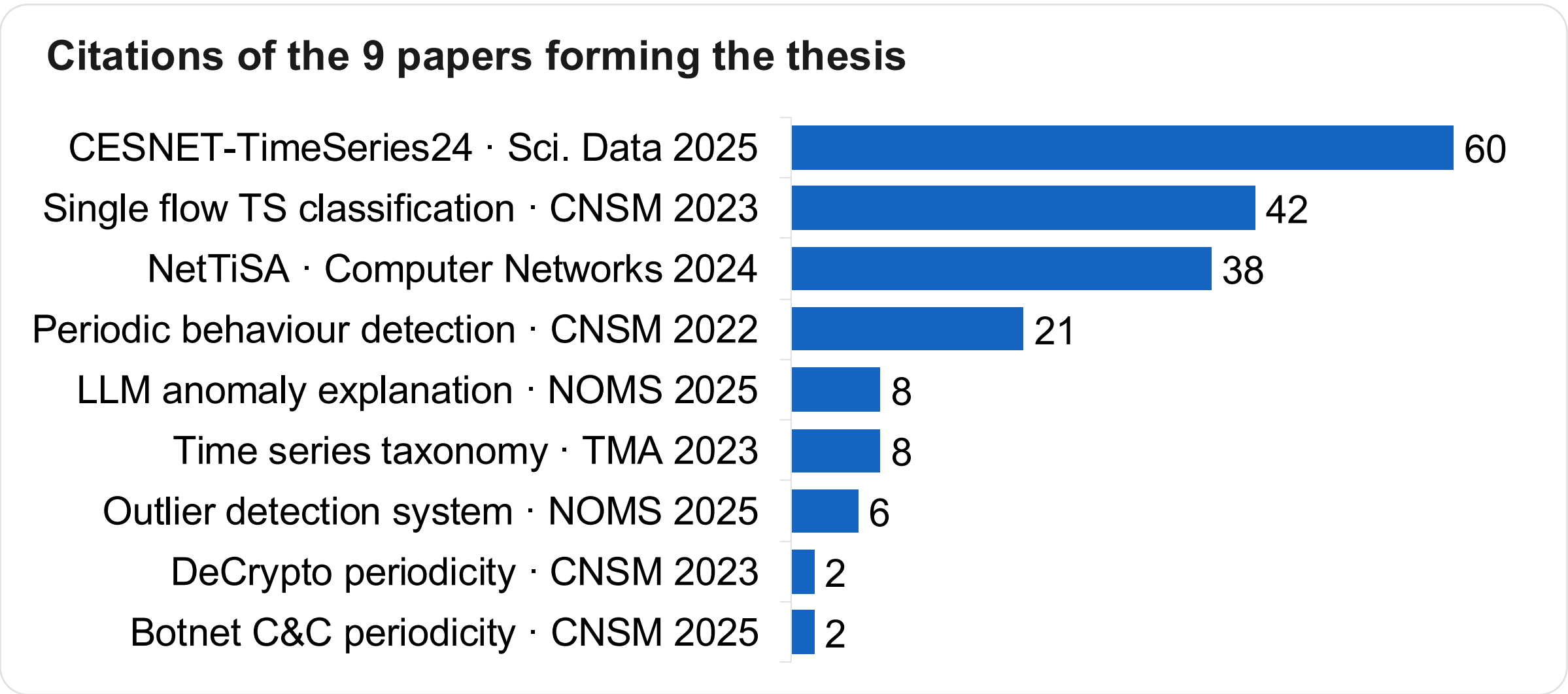
on Zenodo

33,451



dataset downloads

19,127 unique views on Zenodo



Open science

-  Source code released with the papers
-  CESNET-TSZoo library for network traffic time series
-  All data published under the ethics statement approved by the CTU Ethics Committee

Serving the community



Student supervision

- 5 master's students
- 11 bachelor's students
- 7 Summer Research Programme students



Lecturing

- Computer Networks
- Technology of Computer networks
- Hardware Accelerated Network Traffic Monitoring
- 3x NECS PhD school
- 2x BISSIT summer school
- CNSM 2024 PhD school



Conferences

- **2025** TPC member, QoDaNet workshop at CNSM
- **2024** Poster co-chair, CNSM
- **2023–2026** Poster co-chair, PESW



Volunteering

- **2024–2025** Senator in the Academic Senate of FIT CTU
- **2023–2025** Vice-chair of the Commission for the Evaluation of Education Quality of FIT CTU
- **2023–2026** President of the IEEE Student Branch at CTU in Prague



Research projects

- SmartADS TH04010073
- FETA VJ02010024
- Cyberthreats OYCESNET20221
- Open Science II
CZ.02.01.01/00/24_030/0015041

Thank you for your attention

Josef Koumar · Threat Detection in Network Traffic using Time Series Analysis



Papers forming the thesis	Venue	Type	Year	Cit.
Cesnet-timeseries24: Time series dataset for network traffic anomaly detection and forecasting	Nature Scientific Data	Q1 journal	2025	60
Network traffic classification based on single flow time series analysis	CNSM	Core B conf.	2023	42
Nettisa: Extended ip flow with time-series features for universal bandwidth-constrained high-speed network traffic classification	Computer Networks	Q1 journal	2024	38
Network traffic classification based on periodic behavior detection	CNSM	Core B conf.	2022	21
Explainable anomaly detection in network traffic using llm	NOMS	Core B conf.	2025	8
Unevenly spaced time series from network traffic	TMA	Core B conf.	2023	8
Towards building network outlier detection system for network traffic monitoring	NOMS	Core B conf.	2025	6
Enhancing decrypto: Finding cryptocurrency miners based on periodic behavior	CNSM	Core B conf.	2023	2
Botnet Detection Through Periodic Patterns in Command-and-Control Network Traffic	CNSM	Core B conf.	2025	2

Highlighted: Q1 journal papers. Citations from Google Scholar.

Other peer-reviewed publications

Publication	Venue	Type	Year	Cit.
Comparative Analysis of Deep Learning Models for Real-World ISP Network Traffic Forecasting	TNSM	Q1 journal	2026	8
Analysis of statistical distribution changes of input features in network traffic classification domain	NOMS	Core B conf.	2024	6
MFWDD: Model-based feature weight drift detection showcased on TLS and QUIC traffic	CNSM	Core B conf.	2024	4
Augmenting monitoring infrastructure for dynamic software-defined networks	SpliTech	Core B conf.	2023	3
CESNET TS-Zoo: A Library for Reproducible Analysis of Network Traffic Time Series	CNSM	Core B conf.	2025	1
DAF: Device Annotation Framework	CNSM	Core B conf.	2025	2
Device Type Classification on ISP Network Using Time Series Analysis	NOMS	Core B conf.	2026	0
Threat Detection in Network Traffic using Time Series Analysis	NOMS	Core B conf.	2026	0
Is Network Traffic Classification a Solved Problem? Duplication, Ceilings, and the Illusion of Progress	CNSM	Core B conf.	2026	9 (preprint)

Prof. Roberto Magán Carrión

- 1 Single flow – multi-label flows and traffic morphing
- 2 Flow series – lost behaviours, feature distributions, fair comparison
- 3 Aggregated series – choice of windows, labelling of anomalies
- 4 General evaluation – imbalance-aware metrics, MITRE ATT&CK mapping
- 5 Deployability – primary hardware bottleneck

Q **1) Single Flow:** Can a single flow contain multiple simultaneous attack typologies? If so, which label is chosen during the dataset transformation process? Since NetTiSA discards fine-grained packet order, how vulnerable is it to targeted traffic morphing designed to mimic benign statistical distributions?

✓ **Rare in practice, for example, VPN/tunneled traffic or C&C keepalive carrying multiple commands**

- During the packet-to-flow transformation, no multi-label flows were found. This is a result of how these datasets were created. In real-world traffic, such flows will probably occur.
- In occurrence of such flow, the NetTiSA can misclassify. Since one flow gets one label and the behavior of the flow will differ from the average behavior of same label flow. However, this is a general limit of flow-level classification, not specific to the NetTiSA flow. Packet-sequence methods fail here too, even though they keep packet order.

Q

2) Flow Series: You claim critical behavioral patterns are lost when aggregating flows; could you provide concrete examples of these lost behaviors? Why is it impossible to model long-term periodic behaviors using flows?

The description of the dataset structure and features extracted from the Lomb-Scargle periodogram remains unclear. What are the key differences in feature distributions among the various data typologies across your solutions?

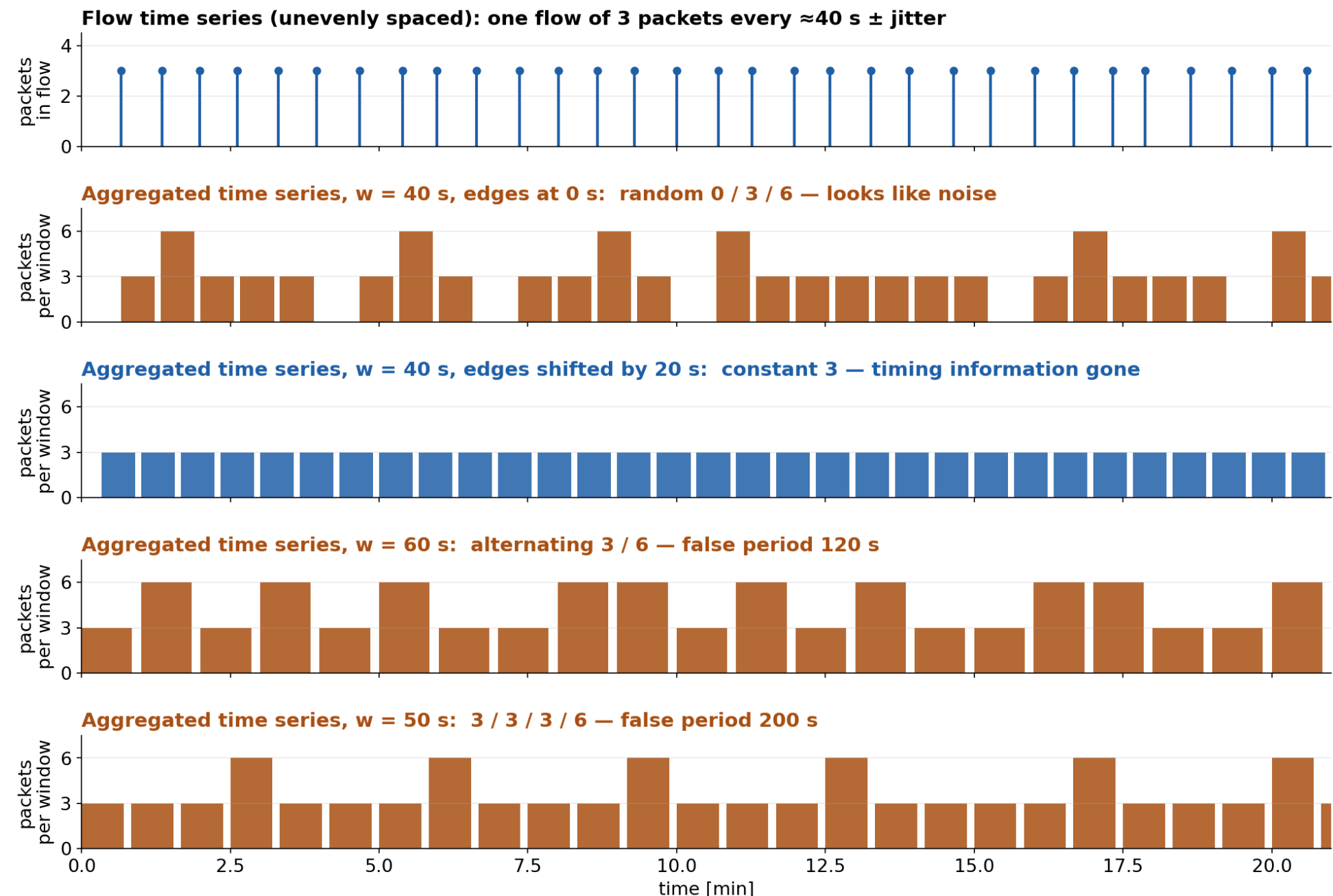
In Table 5.5, you benchmark your solution against external works that utilize completely different training pipelines and ML models. To ensure a fair comparison and rule out external variables, wouldn't a formal Design of Experiments (DoE) and ANOVA—keeping the pipeline identical and altering only the Feature Engineering method—be required?

Which behaviours are lost by aggregation?

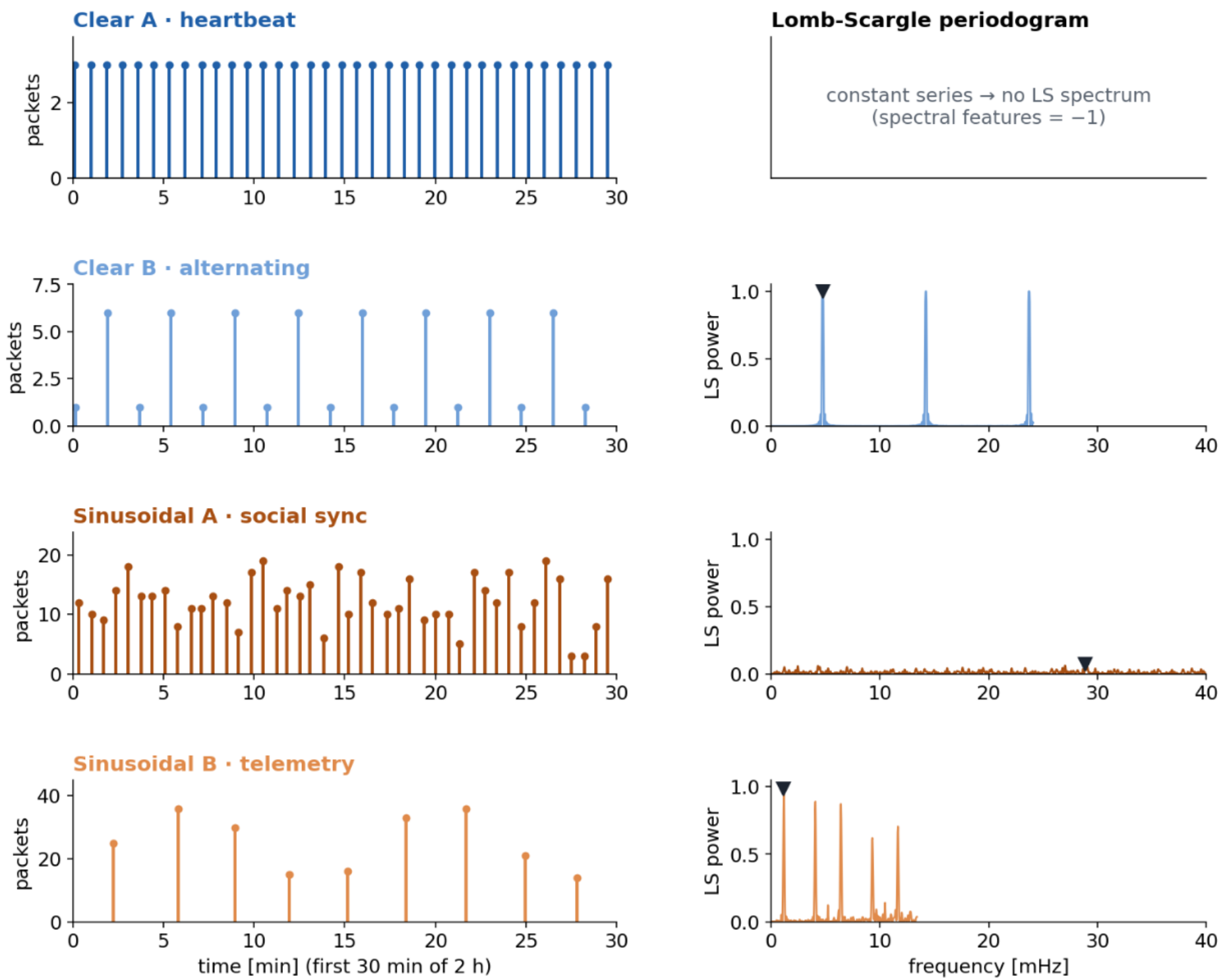
Magán Carrión · Q2a

Q You claim critical behavioral patterns are lost when aggregating flows; could you provide concrete examples of these lost behaviors? Why is it impossible to model long-term periodic behaviors using flows?

- Flows have true period of ≈ 40 s
- The 40 s aggregation window looks like random noise
- Same aggregation shifted by 20 s results constant series
- Aggregation of 50 s and 60 s results in false periods of 200 s and 120 s
- Flow time series keep exact timestamps, so the true period is preserved



Q The description of the dataset structure and features extracted from the Lomb-Scargle periodogram remains unclear. What are the key differences in feature distributions among the various data typologies across your solutions?



feature	Clear A	Clear B	Sin. A	Sin. B
PERIODIC				
packet_value	3	0	0	0
packet_value_x	0	1	11.0	13.0
packet_value_y	0	6	10.0	36.0
bytes_value	180	0	0	0
STATISTICAL				
packet_mean	3	3.5	12.4	25.2
packet_std	0	2.5	3.89	8.63
difftimes_mean [s]	51.7	105	39.9	190
difftimes_std [s]	1.58	0.82	4.25	14.0
LOMB-SCARGLE				
max_power	-1	1	0.07	0.98
max_frequency [mH	-1	4.74	28.9	1.17
spectral_crest	-1	39.8	6.48	18.1
spectral_entropy	-1	21.0	147	49.3

Q In Table 5.5, you benchmark your solution against external works that utilize completely different training pipelines and ML models. To ensure a fair comparison and rule out external variables, wouldn't a formal Design of Experiments (DoE) and ANOVA—keeping the pipeline identical and altering only the Feature Engineering method—be required?

- Ideal comparison would be re-implement all approaches and run evaluation on the same dataset with repeated identical splits and the same test sets, then test statistically.
 - Not a fully identical pipeline, for example scaling
 - Comparison methodology in Table 5.5 compares results on the same dataset (CTU-13), using the same metrics, and unit (multiflow results are transferred to all flows). This is not a controlled comparison, but it is the common practice in this domain. The goal is not to claim higher accuracy, but to show where a multiflow approach stands relative to flow-based methods.
- Our final goal is not replace flow-based methods. The multiflow approach should works alongside flow-based detectors through data fusion, as in the cryptomining case (DeCrypto).

- Q** **3) Aggregated Series:** What was the empirical rationale behind selecting aggregation windows from 10 minutes to 1 hour, and what exact impact does this choice have on detection performance? Could you detail how the expert evaluation and labeling process for the network anomalies was conducted?

Why 10 min and 1 h aggregation windows?

Magán Carrión · Q3a

Q What was the empirical rationale behind selecting aggregation windows from 10 minutes to 1 hour, and what exact impact does this choice have on detection performance?

10 minutes aggregation:

- At ISP line rates (up to 400 Gbps), datapoints are aggregated from IP flows, not packets
- Flow exporter active timeout was 5 min, therefore, long flows are reported in 5-min chunks
- Windows shorter than $2 \times$ active timeout produce artificial spikes and gaps, therefore, 10 min was chosen as the smallest meaningful window for CESNET-TimeSeries24 dataset

1 hour aggregation:

- The most common granularity in SOTA network traffic forecasting
- Preserves daily/weekly seasonality and is human-interpretable

Q What was the empirical rationale behind selecting aggregation windows from 10 minutes to 1 hour, and what exact impact does this choice have on detection performance?

Limitation of our aggregation windows selection process:

- Ideally, the window should be selected per task (domain knowledge or experimental analysis), however, the dataset was created to be for multiple tasks, like network traffic forecasting and anomaly detection in network traffic
- To my knowledge, no study has systematically identified the optimal window for network traffic forecasting.

Impact on performance

- 10 min aggregation has 6 times more datapoints than 1 hour aggregation
- 1-month training window: 4,320 vs 720 datapoints per time series

Complexity of method	Example	10 min vs 1 hour
$O(n)$	MOD, DOD, ETS	6 times slower
$O(n^2)$	DTW, Matrix Profile	36 times slower
$O(n^3)$	Gaussian Processes	216 times slower

How were the anomalies labelled?

Magán Carrión · Q3b

Q Could you detail how the expert evaluation and labeling process for the network anomalies was conducted?

✓ **CESNET-TimeSeries24 is not labeled as a whole because it is not possible to annotate 270k time series across 3 aggregation windows, as annotation is inherently manual work. Therefore, labels were created only for the LLM explainability study with subsample series, and annotated was 70 anomalies in total on 1-hour aggregation.**

Annotation process:

- Pre-selection of significant, clearly pronounced anomalies by **human experts**
- Only anomalies marked by **all** study's domain experts was selected for experiment
- Cross-check with CESNET Warden (security event logs from various detectors) to verify that anomaly in time series means security related behavior
- Final label created by fusion of these information to actionable anomaly (attack, behavioral shift, outage, ...)
- The false positive anomalies were added to test if approach recognize them

- Q** **4) General Evaluation:** Accuracy is a misleading metric for highly imbalanced datasets. Why was the performance not broken down for each specific attack typology to see if the feature engineering fails for certain threat categories? Are these methods capable of mapping the entire progression of an adversary's campaign across frameworks like MITRE ATT&CK or the Cyber Kill Chain?

Q Accuracy is a misleading metric for highly imbalanced datasets. Why was the performance not broken down for each specific attack typology to see if the feature engineering fails for certain threat categories?

✓ **Comparable with SOTA requires presenting Accuracy because most related works report only accuracy, so it is included for comparison, but never on its own.**

- **Imbalance-aware metrics**, F1-score for binary tasks and macro and Weighted and Macro F1-score for multiclass tasks, are used. So every attack class counts equally. Moreover, we used **fair setup**, which including stratified splits, a held-out test set, and models tuned on F1 rather than accuracy.
 - For example, on TON IoT, the best related work has higher accuracy but only 52 % Macro F1-score, while NetTiSA reaches ~83 %
- **Broken down results** per-class for all multiclass datasets are published in the most papers, for example, for NetTiSA it is in paper's appendix.
- Periodicity approach was evaluated **separately per threat category** (Cryptomining and C&C)
- Weak categories identified and explained in texts of paper's, for example, rare classes in UNSW-NB15, Tor vs payload-based methods, and C&C recall on short CTU-13 scenarios.

Q Are these methods capable of mapping the entire progression of an adversary's campaign across frameworks like MITRE ATT&CK or the Cyber Kill Chain?

✓ **The methods detect individual techniques at the network level. They are building blocks for campaign mapping. Therefore, cannot map entire progression of an adversary's campaign on their own.**

Coverage across phases:

- reconnaissance (scan anomalies),
 - credential access (SSH brute force),
 - command & control (beaconing, DoH, Tor), and
 - impact (DoS, cryptomining).
- Detection on network has multiple limits. All **host-level phases** (execution, persistence, privilege escalation) **are invisible** in network telemetry. The evaluation is limited because public datasets label single attacks, not full campaigns.

Q **5) Deployability:** Given that deployment constraints vary by tier (probe, collector, SOC backend), what is the primary hardware bottleneck if your entire architecture were deployed in a unified, un-siloed manner within a real ISP?

✓ **The primary bottleneck would be memory, specifically RAM capacity for per-entity state, rather than CPU or link throughput. The thesis supports this at every tier:**

- NetTiSA add almost no overhead over classic IP flows, so it is not the problem
- Periodicity detection is both memory-intensive (storing large number of dependency time series) and CPU-intensive (due to Lomb-Scargle).
 - Method was further optimized to use Periodogram less often, so the pipeline processes 3 million flows in 5 seconds.
 - Can be deployed in smaller environments or with strict filter per dependencies.
- Anomaly detection has some limitations
 - Aggregating unique metrics (unique destination IPs, ASNs, ports) is memory-intensive, because all unique values must be kept for each IP address
 - Some models for time series forecasting can be resource consuming
 - Detection itself is lightweight
 - Deployment in ISP environment proven by NODS system

REVIEWER QUESTIONS

Prof. Ramin Sadre

Q Long-term performance and the need for retraining

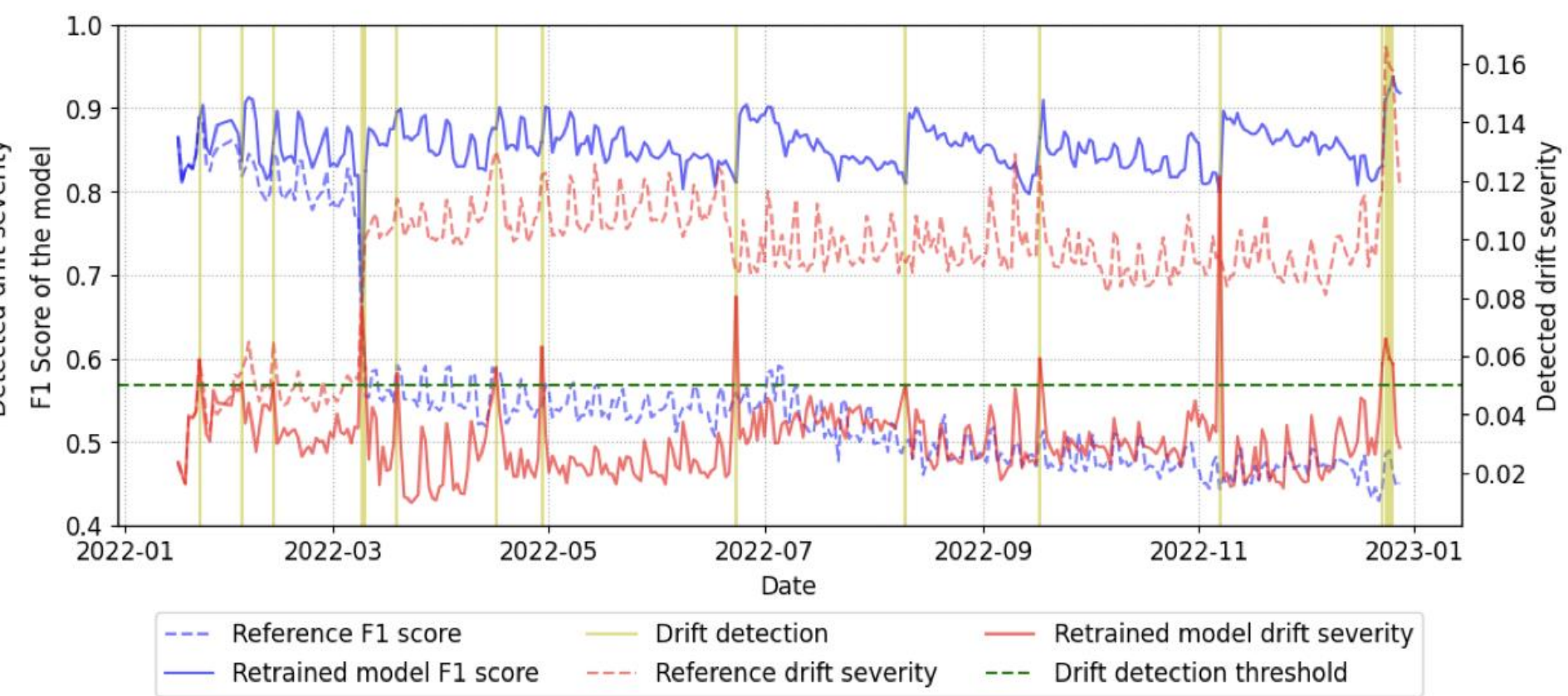
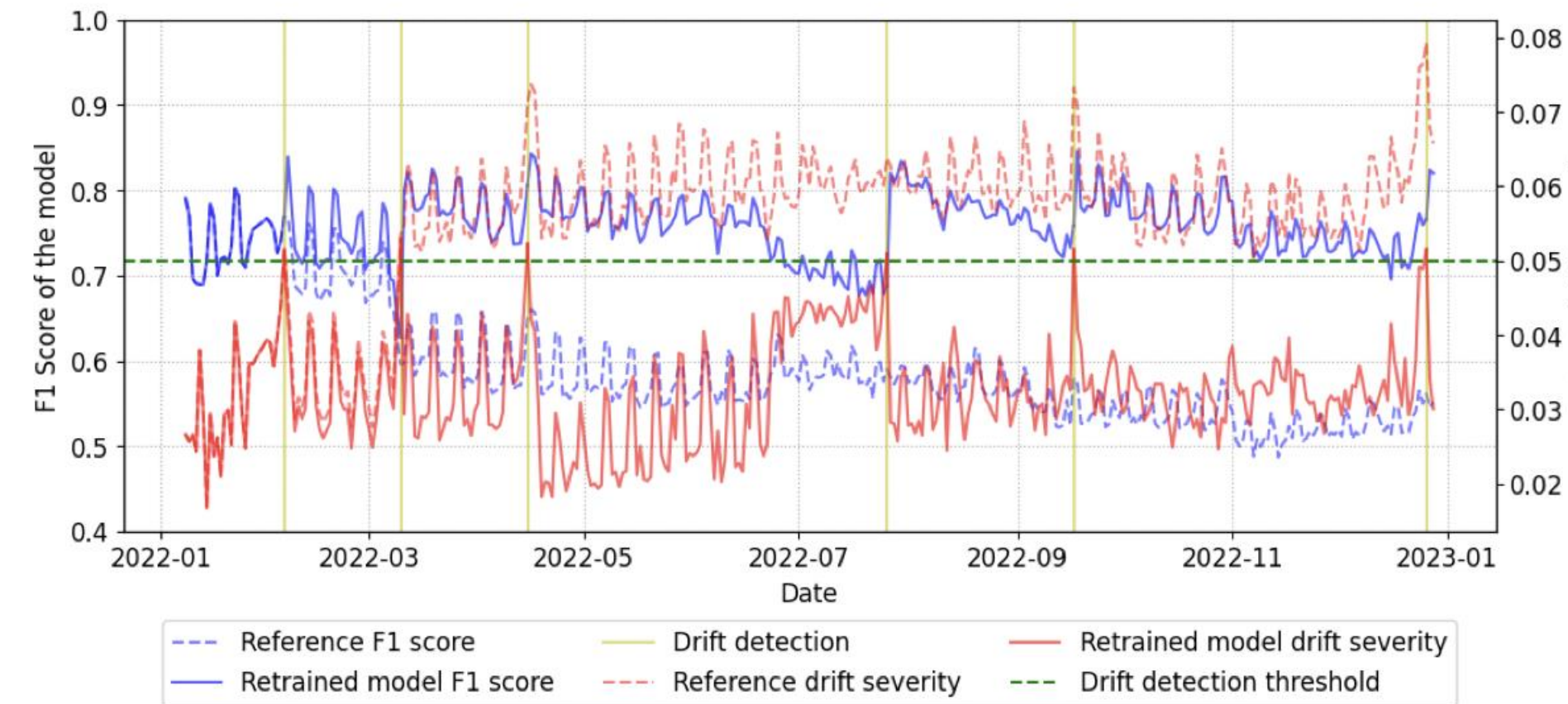
Q Have you looked at the long-term performance of your detection methods? Do you expect that retraining or adaptation of the detection parameters will be necessary after a certain time because the network traffic changes its characteristics?

✓ Yes, for supervised detection method is retraining necessary, but it should be drift-triggered, not periodic

- NetTiSA performance was evaluated over a full year. The MFWDD method (CNSM 2024) was tested on CESNET-TLS-Year22 dataset
 - Drift is real and unavoidable. We found out day/night and weekly periodicity with decreasing trend and possibility of sudden drifts
 - NetTiSA is more stable than raw packet sequences. It needed fewer retrainings.
- Periodicity-based detection, the retraining will also be necessary.
- Forecasting-based methods usually do not require retraining
- For periodicity detection, the threshold is expected to hold

Drift over one year of real traffic

Sadre · Q1



How to read: yellow line = drift detected → model retrained · solid blue = F1 of the retrained model · dashed blue = F1 of the reference model

Assoc. Prof. Pál Varga

- 1 Choosing between the three time series representations
- 2 Robustness to jitter, adaptive timing and mixed traffic
- 3 Are Clear and Sinusoidal classes sufficient?
- 4 Main bottleneck for operational adoption
- 5 Deployability vs. precision in NODS
- 6 Generalisation of the LLM explanation layer

Q 1) The dissertation is built around three traffic-derived representations: Single flow time series, Flow time series, and Aggregated time series. Could you clarify the practical decision rule for choosing among these three in a real deployment - in particular, when would you recommend one representation over the others for new threat classes that were not explicitly studied in the thesis?

✓ They are not alternatives but complementary layers of one SOC detection system. The choice depends on the time horizon and on whether labelled malicious traffic exists.

- Single flow time series (NetTiSA) is used for classification of individual flow records. Usable when the threat is known and **shows itself within one connection**.
- Flow time series (periodicity detection) capture longer-term behaviour across many flows. Use when the threat is defined by **repeated or regular communication** over hours to days.
- Aggregated time series (anomaly detection) are used for modeling the **normal long-term behaviour** of a device or network and flags any inconsistency. It is the only layer that can catch new, **unseen threat classes**, where supervised models fail because no malicious flows are available for training or fine-tuning.

Q 2) The introduced Flow time series methodology relies on detecting periodic structures in sequences of flow arrivals, and it performs very well for threats such as cryptomining and botnet C&C communication. Could you clarify how robust this approach is when the attacker deliberately introduces jitter, adaptive timing, or mixed benign-background traffic to obscure periodicity, and which part of your pipeline you consider the most vulnerable under such evasive behavior?

✓ **The pipeline tolerates moderate jitter and mixed traffic by design. Fully randomized or adaptive timing defeats any statistical threat detector**

- The Lomb–Scargle periodogram is designed for unevenly spaced data. Moderate jitter smaller than the period widens the spectral peak but keeps the fundamental frequency. The Sinusoidal behaviour class captures exactly this kind of noisy periodicity.
- Splitting traffic into network dependencies separates malicious from the host's other traffic, and each Flow time series is checked for multiple periodic behaviours.
 - rotating C&C infrastructure is a weak point because it can split one periodic behaviour into several shorter Flow time series
- In common C&C frameworks, jitter is a simple configuration option, and timing jitter is used far more often than random message padding, although padding is also possible. The detector can be made more robust to both by training with data augmentation that systematically injects timing jitter and random padding.

Are Clear and Sinusoidal classes sufficient?

Varga · Q3

Q 3) In the periodicity-based part of the thesis, you distinguish Clear and Sinusoidal periodic behaviors and build the detection pipeline around Lomb–Scargle analysis and significance testing for unevenly sampled Flow time series. Could you clarify whether you see these two behavior classes as sufficient for most operationally relevant threats, or whether there are important traffic patterns that fall outside this model and would require extending the periodicity framework?

✓ **The two classes describe the volume dimension of periodic communication. The constant volume, Clear periodic behavior, represent traffic such as heartbeats, C&C, and cryptomining polling. The varying volume, Sinusoidal periodic behavior, represent other periodic signals such as synchronization. The timing and volume dimension is handled by the Lomb–Scargle periodogram itself.**

- Multiple and nested periodicities are handled to ensure method working in such cases.
- Threats without multiflow periodic behavior are not covered by this method.

Q 4) A strong theme of the thesis is deployability in ISP-scale networks. NetTiSA as well as the periodicity-detection pipeline are presented as deployment-oriented solutions. Could you clarify what you see as the main remaining bottleneck for broader operational adoption: collector overhead, feature extraction cost, long-term state maintenance, false-positive handling, analyst workflow integration, or others?

✓ **The primary bottleneck would be memory, specifically RAM capacity for per-entity state, rather than CPU or link throughput. The thesis supports this at every tier:**

- NetTiSA add almost no overhead over classic IP flows, so it is not the problem
- Periodicity detection is both memory-intensive (storing large number of dependency time series) and CPU-intensive (due to Lomb-Scargle).
 - Method was further optimized to use Periodogram less often, so the pipeline processes 3 million flows in 5 seconds.
 - Can be deployed in smaller environments or with strict filter per dependencies.
- Anomaly detection has some limitations
 - Aggregating unique metrics (unique destination IPs, ASNs, ports) is memory-intensive, because all unique values must be kept for each IP address
 - Some models for time series forecasting can be resource consuming
 - Detection itself is lightweight
 - Deployment in ISP environment proven by NODS system

- Q** 4) A strong theme of the thesis is deployability in ISP-scale networks. NetTiSA as well as the periodicity-detection pipeline are presented as deployment-oriented solutions. Could you clarify what you see as the main remaining bottleneck for broader operational adoption: collector overhead, feature extraction cost, long-term state maintenance, false-positive handling, analyst workflow integration, or others?

Traffic drift is real and unavoidable which is challenging for model maintenance:

- Supervised models (NetTiSA, the periodicity classifier) need retraining, triggered by detected drift rather than on a fixed schedule. NetTiSA proved more stable than raw packet sequences over a full year.
- Forecasting-based anomaly detection and the periodicity thresholds are expected to hold.
- At ISP scale is the key operational challenge the False-positive handling. Detector with 99.99% of Accuracy produce at ISP scale still too much False positives in one day. Therefore, no single method is enough. Detections should be combined through data fusion across all three layers. Confidence can be raised with external and unencrypted context, such as blocklists, threat intelligence, DNS, and TLS SNI or fingerprints.
- Analysts need a single compromise score per device instead of raw alerts, and for critical score analyst will deep dive into the logs. The LLMs can help with that.

Q 5) In the NODS anomaly-detection system, the models are chosen partly because they are time- and memory-sensitive enough for deployment, even if this means prioritizing deployability over precision. Could you clarify how you would balance this trade-off in a production SoC setting, especially when the goal is not only to detect outliers but to keep alert volumes manageable for analysts?

✓ Precision does not have to come from the individual models. Lightweight models provide broad coverage of all monitored IP addresses, and precision is built in the later stages of the pipeline:

- MOD and DOD are fast and memory-light, so they scale to 100,000 IP addresses. The more expensive SARIMA or Foundation models like Chronos or TimesFM can be applied selectively to critical assets such as servers.
- Outliers are grouped into incidents across models, metrics, and aggregation intervals (10 min and 1 h). Only critical incidents, where all models agree in both intervals, are sent to the SOC. This reduced alert volume to a level an average SOC team can handle, while the other incidents are logged for later analysis.
- The LLM layer prioritizes and explains incidents. Enrichment with blocklists and threat intelligence raises confidence further.

- Q** 6) The LLM-based component is positioned as a layer that explains anomalies already detected by a prior anomaly-detection framework, using a zero-shot setup rather than domain-specific fine-tuning. Could you clarify how far you believe this approach can generalize across networks and operational contexts, and what would be the most important next step to move it from a promising proof of concept toward a robust production-grade analyst assistant?

Why the model would generalize across networks and operation contexts:

- The LLM models are trained on huge number of sources, including security and network related sources. Thus, knowledge of LLM models are general enough.
- The input contains generic volumetric metrics, their statistics over the preceding 48 hours and 7 days, and time context (time of day, weekday, public holidays).

Most important next step toward a robust deployment:

- Not all anomalies can be process in real time → the priority threshold should be calibrated in real world deployment
- Enrich the context with retrieval of topology, asset inventory, threat intelligence, and past incidents
- Use open-source models running on-premises, for data privacy and reproducibility
- Keep it as an analyst assistant with human oversight rather than an autonomous decision-maker